

**RL for CXL Memory Pooling:
When Learning Helps, When Greedy Wins**

Pulak Mehrotra

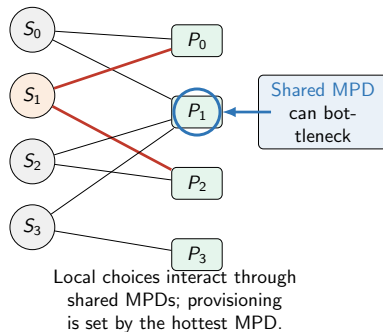
May 2026

Why RL was worth trying: greedy is local, the objective is global.

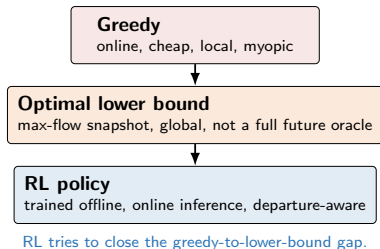
Systems problem

Each VM arrives online and must be placed on one reachable CXL Memory Pool Device (MPD). The action is local, but the cost is the **peak MPD load over the full trace**.

- **Greedy:** pick the least-loaded reachable MPD now.
- **Failure mode:** filling a shared MPD can remove headroom for a different host later.
- **RL hypothesis:** learn when current load is transient versus sticky from trace replay.



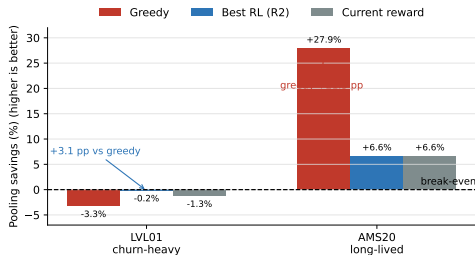
Greedy is deployable; optimal is a benchmark; RL tries to close the gap.



- ▶ Greedy is the right baseline because it can run at every VM arrival.
- ▶ The exact solver computes a tight per-snapshot lower bound $t^*(s)$ using Dinic max-flow.
- ▶ The lower bound is diagnostic: it shows whether the current state wasted peak capacity.
- ▶ RL is only worth considering if trace structure creates a repeatable policy better than local load balancing.

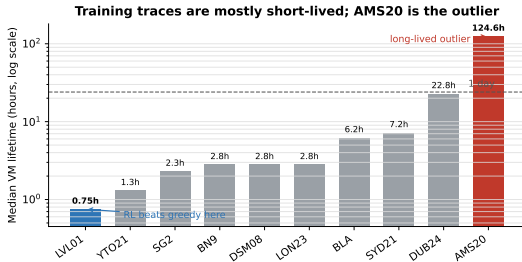
Careful claim: “optimal” is a lower bound and shaping signal, not a deployable 14-day oracle.

Best result: RL helps on churn-heavy traces, but generalization is not solved.



- **LVL01:** best RL (R2) reaches -0.2% savings vs. greedy -3.3% .
- This is a **+3.1 percentage point** improvement, nearly break-even.
- **AMS20:** greedy wins decisively: $+27.9\%$ savings vs. RL variants around $+6.6\%$.
- This is a workload-regime split, not a universal RL win.

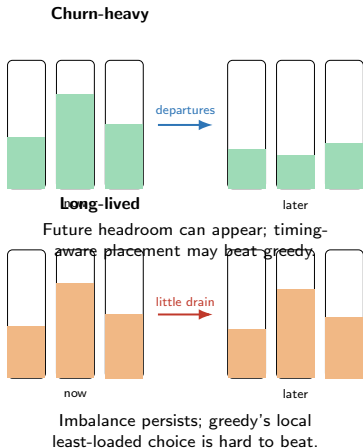
The policy generalized to the lifetime regime it mostly saw.



- The multi-trace training pool is mostly short-lived; most median lifetimes are under a day.
- **LVL01** is the churn-heavy regime where RL beat greedy: median **0.75h**, **56.7%** of VMs < 1h.
- **AMS20** is the long-lived outlier: median **124.6h**, **49.3%** of VMs > 7d.

Sticky load explains why lifetime regime changes the best policy.

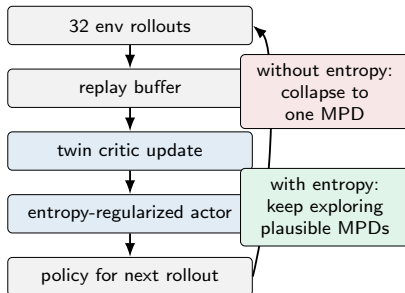
- ▶ **Transient load:**
short-lived VMs drain soon, so a full MPD can become useful again.
- ▶ **Sticky load:** long-lived VMs stay resident, so today's imbalance becomes tomorrow's bottleneck.
- ▶ Greedy is strong when load is sticky: the least-loaded reachable MPD is often the right local choice. a currently-full MPD the better future choice.



This is why the same policy can help on LVL01 but fail to generalize to AMS20.

SAC was chosen because collapse was the first failure mode.

- ▶ Early policies collapsed to one or two MPDs even when load changed.
- ▶ SAC adds an entropy term, keeping the policy from becoming deterministic too early.
- ▶ Off-policy replay fits trace replay: one allocation experience can support many gradient updates.
- ▶ SB3 SAC worked cleanly with 32 parallel environments, which made controlled ablations practical.



I avoid PPO-specific claims here; the documented comparison is against LCBO and A2C

Augmentation was domain randomization, not random noise.

- ▶ VM memory sizes are SKU-like; per-VM scaling creates fake VM types.
- ▶ So randomization happens at the **episode** level.
- ▶ Enabled knobs: trace ID, pod seed, episode memory scale.
- ▶ Disabled until validated: arrival jitter, lifetime perturbation, link failures.

episode augmentation

• trace ID

• pod seed

• memory scale

○ arrival jitter

○ lifetime perturbation

○ link failures

HOTFIX survival by scale:

0.7 → 83.6% 1.0 → 81.0%

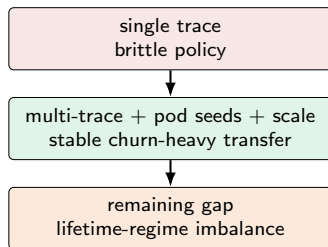
1.2 → 76.7% 1.5 → 64.9%

Scale range

Use $s \sim \text{Tri}(0.7, 1.0, 1.2)$: enough load diversity, without making HOTFIX dropout dominate the episode.

Augmentation improved robustness, but not lifetime-distribution shift.

- ▶ Single-trace training overfit and was brittle on unseen traces.
- ▶ Multi-trace + pod seed + scale augmentation made stable held-out evaluation possible.
- ▶ Training savings improved from run7's **~1.3%** plateau to **~5%** in later multi-trace ablations.
- ▶ That comparison is confounded by skip-hotfix, so the strongest claim is robustness, not a clean causal win.



Trace contrast:

AMS20: median 124.6h, 49.3% > 7d

LVL01: median 0.75h, 56.7% < 1h

Proxy rewards teach every placement; terminal reward grades the whole trace.

Proxy reward

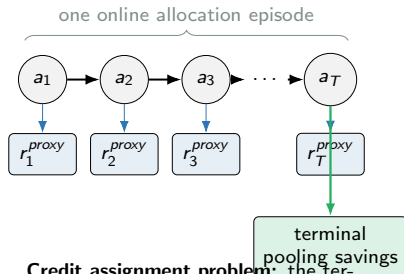
A local signal after each VM placement: did this action reduce immediate pressure, imbalance, or an estimated optimality gap?

Terminal reward

The actual objective after replaying the trace:

$$\frac{\text{greedy peak} - \text{policy peak}}{\text{greedy peak}}.$$

Design tension: dense feedback is learnable but can be misaligned; exact feedback is aligned but arrives too late for easy credit assignment.



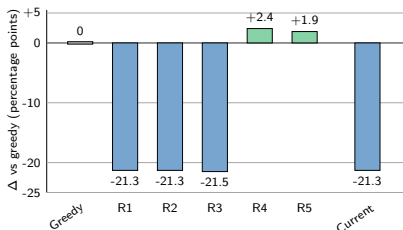
Credit assignment problem: the terminal score says whether the trace was good, but not which early placement preserved or burned future headroom.

Reward design aligns local decisions with the global peak objective.

- ▶ **Proxy rewards:** dense and learnable, but can optimize the wrong thing.
- ▶ **Terminal reward:** exactly pooling savings, but sparse over thousands of decisions.
- ▶ **Optimal gap:** max-flow lower bound; principled but expensive.
- ▶ **Takeaway:** reward design changes the workload regime where RL helps, not just the training curve.

Measurement problem: which scalar signal faithfully reflects the peak-memory objective at each local decision?

All reward ablations compared to greedy.

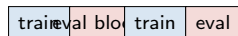


Bars show saved held-out RL performance minus the matching greedy baseline. R1–R3/current use AMS20; R4–R5 use LVL01.

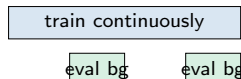
The systems redesign turned reward ablation from days into hours.

Async eval worker

Old:



New:



44 min of blocked evaluation per
2M-step run eliminated.

Throughput

Configuration	fps
Run7 full training baseline	53
Pre-speedup env-only	1,009
Post-speedup env-only	2,757

- ▶ Flat MPD arrays replace list-of-lists hot loops.
- ▶ Multi-trace precompute cache makes resets cheap.
- ▶ Numba compiles departure and sticky-load kernels.

Full-training and env-only fps are labeled separately.

What the experiments actually say.

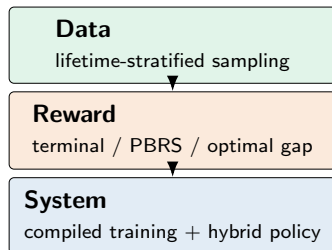
Where RL helps	Where greedy wins	Next bottleneck
High-churn workloads where departure timing changes the right MPD choice.	Long-lived stable load where the least-loaded local MPD is already a strong rule.	Distribution-aware generalization across VM lifetime regimes.

- ▶ RL is not automatically better than greedy; it helps when trace structure rewards departure-aware placement.
- ▶ Reward shaping made the problem measurable, but did not by itself solve cross-trace transfer.
- ▶ The simulator is now fast enough to answer the next questions with controlled ablations.

Future work: make the learned policy robust, not just better on one regime.

- ▶ Balance training batches by lifetime regime so AMS20-like traces are not underrepresented – figure out non-stationarity.
- ▶ Does optimal help with learning?
- ▶ Expand augmentation only with validated knobs: arrival jitter, lifetime perturbation, topology/link failures.

Deployment framing: keep greedy as a fallback; use RL only where departure-aware placement is justified.



Appendix

Extra implementation details and diagnostic backup slides.

Backup: what was built.

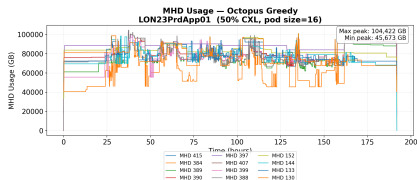
Simulator and state	RL and reward study
Gymnasium CXL memory-pooling env; online VM-arrival replay; reachable-MPD action constraints; departure-aware state with D_j , S_j , P_j , and Q_j .	SB3 SAC primary path; LCPO/A2C explored; reward ablation R1–R5 from local proxy to terminal/PBRS objective-aligned signals.
Training data	Systems engineering
Ten Azure traces characterized by VM size, arrival rate, lifetime, and pod density; 7 train / 1 val / 2 held-out split; episode-level augmentation.	Async eval worker removes blocked callbacks; flat MPD arrays, multi-trace cache, and Numba raise env-only throughput to 2,757 fps; JAX trainer built as compiled alternative.

Backup: Greedy packs load onto a few MPDs; RL spreads it.

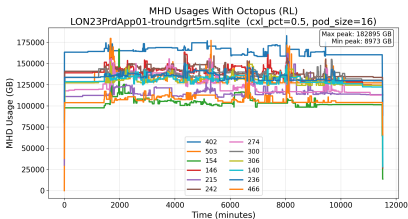
Greedy

ratio ≈ 6.1 RL (run 7)

ratio ≈ 1.04



A few MPDs saturate; most stay idle.



Load spread across all MPDs; no bottleneck.

LON23 validation trace, CXL fraction 0.5, pod of 16 nodes. Each line = one MPD over the replay window.

Backup: SAC objective and why entropy mattered.

Soft Actor-Critic objective

$$J(\pi) = \mathbb{E} \left[\sum_t \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

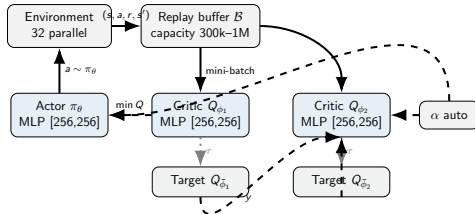
The extra entropy term rewards the policy for keeping probability mass across multiple plausible MPDs instead of collapsing immediately to a single deterministic placement.

- ▶ r_t : reward from the placement objective.
- ▶ $\gamma = 0.999$: long horizon because trace episodes have thousands of VM arrivals.
- ▶ α : entropy temperature, auto-tuned by SB3.
- ▶ $\mathcal{H}$: exploration pressure; low entropy means premature collapse.

Project-specific reason

Early rewards produced collapsed policies. SAC made collapse visible through `sac/ent_coef` and gave the actor pressure to keep exploring.

Backup: SAC architecture and diagnostics.



- Replay buffer reuses trace-placement experience.
- Twin critics use $\min(Q_1, Q_2)$ to suppress overestimation.
- Actor maximizes value while entropy prevents early deterministic collapse.
- Auto- α tunes exploration pressure; logged as `ent_coef`.
- Health checks: critic loss, Q-values,